

# ARA and Cabbage, a patch made in heaven?

Rory Walsh\*

Dundalk Institute of Technology [rory.walsh@dkit.ie](mailto:rory.walsh@dkit.ie)

**Abstract.** The Audio Random Access (ARA) extension has significantly expanded the capabilities of modern music production by allowing plugins access to entire audio regions. While highly powerful, developing ARA-capable plugins typically requires navigating dense, low-level APIs, a substantial barrier for audio researchers and open-source creators. This paper introduces a new ARA implementation within Cabbage 3<sup>1</sup>. It focuses on the design patterns utilised within the core processing backend to negotiate host-to-plugin asset delivery, thread-safe memory sharing, and time-stamped playback markers within plugin environments. Furthermore, it explores how Cabbage 3 abstracts these low-level ARA callbacks, exposing the raw audio data and region metadata through dedicated Csound opcodes. This gives developers full access to the session timeline and audio assets using standard Csound syntax, enabling custom analysis and processing workflows without requiring complex ARA SDK knowledge.

**Keywords:** Csound, Cabbage, ARA, Audio Random Access, Audio Plugins, CLAP

## 1 Introduction

The development of modern audio effects and digital audio workstation (DAW) integration has traditionally relied on plugin APIs such as VST3, AU, and CLAP [1] that stream data to the plugins in blocks of samples. While highly effective for real-time, these architectures present structural bottlenecks for algorithms requiring non-linear access to audio assets. To address this limitation, the Audio Random Access (ARA) extension was introduced, bridging the architectural gap by granting plugins immediate, non-linear random access to entire underlying audio assets and timeline structural configurations directly from the host DAW.

Despite its obvious utility, creating ARA-capable plugins remains a difficult task. The official ARA SDK demands deep, exhaustive mastery of low-level concurrency mechanisms, asynchronous host-to-plugin transaction models, and rigid memory-management lifecycles [2]. For audio researchers, and computer music composers who rely on accessible, high-level programming environments like Csound [3], the implementation overhead represents a real barrier to entry. Furthermore, academic music technology literature has tended to overlook ARA-centric plugin architectures, leaving a gap between commercial software and open-source research [4].

This paper introduces native ARA integration within the Cabbage 3 development framework. By implementing a centralized processing engine and exposing raw asset audio and timeline metadata through an intuitive opcode suite, Cabbage 3 entirely abstracts the low-level ARA API. The following sections outline the thread-isolated asynchronous architecture, detail the structure of the new Csound opcodes, and demonstrate how developers can deploy non-linear audio processing routines using standard orchestra syntax.

## 2 Core Backend Architecture & Asynchronous Pipeline

The Cabbage 3 ARA implementation separates real-time timeline tracking from audio file reading. Structural details, such as region positions, source properties, and user selection states, are updated instantly via ARA callbacks and exposed through a thread-safe JSON store. Meanwhile, the heavy lifting of loading and buffering the raw audio samples is offloaded to a background worker thread, keeping the primary real-time audio loop entirely uninterrupted [5].

---

\* Victor Lazzarini, Steven Yi

<sup>1</sup> The source code is available at <https://github.com/rorywalsh/cabbage3>.

## 2.1 Lifecycle Tracking and Thread-Safe Data Access

Host DAW timeline modifications and asset updates are intercepted directly via the framework's ARA document controller layer. When an asset modification event occurs, the engine schedules it for processing by waking a dedicated background worker thread. This worker reads the audio data via native ARA host readers and caches the raw sample data inside a global, thread-safe singleton (`ARADataPool`) that is shared and accessible across all running plugin instances.

To notify the real-time audio thread that fresh data is available, a lightweight counter is incremented on the non-real-time idle thread whenever the ARA document model changes. It is exposed to Csound through the k-rate opcode `cabbageAraGetUpdate` and intended solely as a trigger to the other ARA opcodes. The ARA opcodes do not acquire the pool mutex or copy state on each k-cycle: `cabbageAraGet` is i-rate only and only queries the central JSON pool on demand. On top of this, it only returns the requested property, and not the full JSON object.

## 3 Csound Abstraction Layer and Custom Opcodes

The Cabbage 3 ARA implementation uses the Csound Plugin Opcode Framework[6] to implement native Csound opcodes rather than generic software channels. By wrapping the timeline data queries inside dedicated opcodes, the engine eliminates the memory overhead of maintaining large fixed-boundary channel arrays and completely bypasses the need for manual cross-instance index matching on the orchestra side.

### 3.1 Opcode Registration and Context Insulation

During processor instantiation on the main thread, the Cabbage backend registers a suite of proprietary Csound opcodes tailored for the ARA timeline context. When a Csound instrument invokes a Cabbage ARA opcode, the underlying C++ code handles index validation safely out of reach of the orchestra heap. If host initialisation delays occur, the opcodes return sensible defaults (such as zero values or empty strings) until the host assigns the specific playback renderer timeline regions to the instance, ensuring absolute state isolation and preventing execution-loop crashes.

### 3.2 The Custom ARA Opcode Suite

Data extraction is organised into four focused opcodes that map directly to the underlying data structures: a consolidated metadata reader, an update trigger, a PCM sample accessor, and a diagnostic dump.

**Consolidated Metadata Access** The primary opcode, `cabbageAraGet`, reads from a thread-safe JSON data store that is updated automatically whenever the host's document model changes. It supports both numeric and string output, and can query top-level state, per-source metadata, playback region properties, and editor view selection data. Property names use dot notation to express the data hierarchy:

- `val:i=cabbageAraGet("property")`: Reads global, top-level framework states such as change counters, event logs, or total asset counts.
- `val:i=cabbageAraGet("property", iIndex)`: Reads specific structural metadata and region timing metrics from the central data pool using the asset's source index.
- `val:S=cabbageAraGet("property")`: A string-returning variant used explicitly for retrieving source names and serialized JSON array configurations.

ARA properties use two time domains. Source-relative properties (crop position within the audio file) are available in both seconds and samples at the source's sample rate. Arrangement properties (where a clip sits on the host timeline) are given in seconds only, since the host's playback position is based on the host sample rate which can change, whereas the clip position does not.

The JSON store is rebuilt incrementally when sources are added or regions are updated, ensuring that i-rate queries during Csound initialization always reflect the latest session state. Dot-notation traversal provides access to the full document hierarchy, including playback parameters (`"playbackRegion.name"`, `"playbackRegion.start"`, `"playbackRegion.duration"`) and editor view selection data (`"editorView.selectedRegion.name"`, `"editorView.selectedPlayback.start"`, `"editorView.timeRange.start"`).

## Update Notifications

- `cabbageAraGetUpdate (k)`: Returns a counter that increments on every ARA lifecycle event (region edits, source updates, edit cycles). Use with Csound's `changed()` to detect new activity.
- `cabbageAraGetUpdateEvent (S[,k])`: Returns both the counter and an optional string identifying the most recent event type.
- `cabbageAraDump`: Diagnostic opcode that prints the complete ARA state to the Csound output window. Supports both i-rate (dump once at init) and k-rate with a trigger input (dump on 0-to-nonzero transitions).

**Raw Sample Access** Four variants provide direct access to the ingested PCM data, supporting different rate contexts and transfer sizes:

- `out:a=cabbageAraGetSourceSamples(aPos,iChan,iSourceIdx)`: Audio-rate sample reader that streams a block of audio samples per k-cycle at the specified position.
- `out:k=cabbageAraGetSourceSamples(kPos,iChan,iSourceIdx)`: Control-rate variant for querying discrete samples at a specific timeline position.
- `arr:k[]=cabbageAraGetSourceSamples(iStart,iCount,iChan,iSourceIdx)`: Control-rate array reader
- `arr:i[]=cabbageAraGetSourceSamples(iStart,iCount,iChan,iSourceIdx)`: Initialization-rate array reader

### 3.3 Instance Verification and Change Triggers

To maintain state consistency across multi-track layouts sharing a single data pool, the engine performs explicit context verification. This addresses asynchronous timing lag common during DAW project loads, where a plugin instance may not immediately be assigned its timeline regions by the host. The processor calls internal methods to identify its specific clip index; if regions have not yet been assigned, a name-matching fallback resolves the asset identity against the global tracking structure to publish the valid context dynamically. When the host updates playback region properties (e.g., resizing or moving a region), the `araPlaybackRegionPropertiesUpdated` callback writes the new start and duration into the data pool, and an increment to the `araUpdateCounter` is triggered, serving as a cross-thread trigger for the Csound orchestra (via `cabbageAraGetUpdate` and `changed()`). This allows the Csound orchestra to safely evaluate the updated region data on its own thread.

## 4 Practical Use Cases and Applications

With the introduction of these native opcodes, ARA integration is made possible within a Csound orchestra for the first time. Because the underlying engine completely abstracts the complexities of thread synchronization, host timeline updates, and global data pool management, developers can deploy full ARA workflows directly through standard, high-level opcode signatures.

### 4.1 Implementation Archetype: Non-Linear Timeline Reversal

To demonstrate the framework's modern opcode API, consider a real-time playback-reversal plugin. Traditional real-time streaming plugins cannot achieve this because they lack access to the entire timeline. With ARA support this implementation becomes trivial. One can simply use a negative phase frequency to produce a signal that moves from `numSamples` to 0:

```
instr AraListener
  updated:k = cabbageAraGetUpdate()
  if changed:k(updated) == 1 then
    event "i", "ProcessData", 0, 3600
  endif
endin

instr ProcessData
```

```

index:i = cabbageAraGet("currentIndex")
numSamples:i = cabbageAraGet("audioSource.sampleCount", index)
phase = phasor(sr / numSamples)
signal:a = cabbageAraGetSourceSamples(-phase*numSamples, 0, index)
out(signal, signal)
endin

```

Because the opcodes expose raw PCM data and region metadata directly, developers are free to implement any analysis or processing algorithm they require, from simple gain staging to complex spectral transformations, entirely within standard Csound syntax.

## 4.2 Rapid Prototyping and IDE-Centric Workflows

A key operational advantage of this opcode-centric architecture is its decoupling of the DSP asset lifecycle from the compiler target layer. Traditional ARA plugin development requires complex toolchains, rigid test-host frameworks, and frequent DAW reboots to evaluate backend changes [2].

Because Cabbage separates the compiled binary wrapper from the functional Csound asset script, developers can build, debug, and validate ARA behaviors entirely within Visual Studio Code (VS Code). A top-level 'ara' object can be used to load audio files into the global ARA pool. This allows for rapid iteration loops on custom analysis algorithms without requiring binary redeployment or manual DAW testing.

## 4.3 Implementation Remarks & Host Compatibility

The current engine implementation was developed and evaluated primarily within REAPER. While REAPER provides a rigorous testing bed for ARA document controller interactions, complete cross-host compatibility remains ongoing work. Because individual DAWs implement host-side ARA specifications with subtle variations, comprehensive testing across additional hosts (such as Logic Pro and Studio One) is necessary to ensure production-level robustness across all environments.

## 5 Conclusion

This paper has presented the integration of the Audio Random Access (ARA) extension within Cabbage 3, establishing a definitive bridge between low-level timeline modifications and high-level Csound code. By encapsulating non-linear audio reading routines inside an isolated background worker thread and managing memory through a thread-safe, global `ARADataPool` singleton, the backend completely insulates the high-priority real-time audio thread from blocking. [5].

The introduction of the specialized ARA opcode suite natively unlocks capabilities previously inaccessible within audio domain-specific languages. By handling complex string parsing, array serialization, and multi-track instance verification out of reach of the orchestra heap, Cabbage 3 transforms a dense, highly specialized C++ SDK into a series of intuitive, declarative syntax queries. The resulting framework vastly lowers the technical barrier to entry for open-source creators and researchers [4], driving rapid prototyping workflows and allowing developers to engineer innovative, non-linear DSP strategies entirely within the native Csound ecosystem [3].

## References

1. Free-Audio Consortium, *CLAP (CLever Audio Plug-in) Specification: Core ABI and Extensions*, Version 1.2.8, 2026. [Online]. Available: <https://github.com/free-audio/clap>
2. S. Gretscher, "10 Things Every ARA Programmer Should Know," presented at the *Audio Developer Conference (ADC 2022)*, London, UK, Nov. 2022. [Online Video]. Available: <https://www.youtube.com/watch?v=xEW8w11ER4A>
3. V. Lazzarini, J. fitch, S. Yi, J. Heintz, Ø. Brandtsegg, and I. McCurdy, *Csound: A Sound and Music Computing System*, Berlin, Germany: Springer, 2016.
4. Ø. Brandtsegg, "A Toolkit for Experimentation with Signal Interaction," in *Proceedings of the 18th International Conference on Digital Audio Effects (DAFx-15)*, Trondheim, Norway, Dec. 2015.

5. R. Bencina, “Interfacing Real-Time Audio and File I/O,” in *Proceedings of the 2014 Australasian Computer Music Conference (ACMC 2014)*, Melbourne, Australia, 2014, pp. 17–23.
6. V. Lazzarini, “Supporting an Object-Oriented Approach to Unit Generator Development: The Csound Plugin Opcode Framework,” *Applied Sciences*, vol. 7, no. 10, p. 970, Oct. 2017. <https://doi.org/10.3390/app7100970>